

Fast Einsum With Batch Matrix Multiplication

Algorithm Engineering 2025 Project Paper

Erik Henicke

Friedrich Schiller University Jena

Germany

erik.henicke@uni-jena.de

Sonja Weitzing

Friedrich Schiller University Jena

Germany

sonja.marina.weitzing@uni-jena.de

ABSTRACT

Einstein summation provides a clear and concise way to perform many multi-dimensional, linear algebraic array operations. However, the einsum implementation in NumPy still leaves room for improvements. We show that by using batch matrix multiplication the einsum implementation can be sped up significantly (Fig. 1). We further study how different enhancement techniques like kernelization, blocking and parallelization effect the performance of our implementation. The einsum library we provide can be utilized in any project benefiting from fast and efficient linear algebraic operations.

KEYWORDS

Einsum, Batch Matrix Multiplication, Transpose-Transpose-GEMM-Transpose, Transposition, Tensor Computations, Vector Intrinsic

1 INTRODUCTION

Einsum is a versatile tool for many multi-linear tensor operations made popular by NumPy’s implementation in 2011. Einsum has a great expressive power and is a part of major machine learning frameworks such as PyTorch and TensorFlow, leading to its widespread use in deep learning [4, 5]. Since large parts of deep learning are series of matrix multiplications and tensor contractions, they can be easily mapped to einsum expressions. Other applications include fast computation of probabilistic circuits [3], and simulation of quantum circuits [12]. Because of its wide application, speeding up NumPy’s einsum implementation is highly desirable.

Our approach to improving einsums’s performance is to express any tensor contraction in terms of batch matrix multiplication (BMM). For the translation between tensors and matrices we rely on the einsum implementation from the repository `einsum_bmm` by Johnnie Gray. The `einsum_bmm` project is licensed under GPL-3.0, which we incorporate into our project. Our main work focuses on implementing a fast BMM in C++ and evaluating it in a comprehensive benchmark.

1.1 Background

The approach of reducing tensor contractions to BMM is known as Transpose-Transpose-GEMM-Transpose or TTGT [9], with the only difference that batch dimensions may be present. This allows us to generalize over all contracting einsum expressions. If no contracting dimension is present, simple element-wise multiplication can be employed.

For this approach, the input tensors must be transposed and reshaped so that they can be interpreted as batches of matrices. The

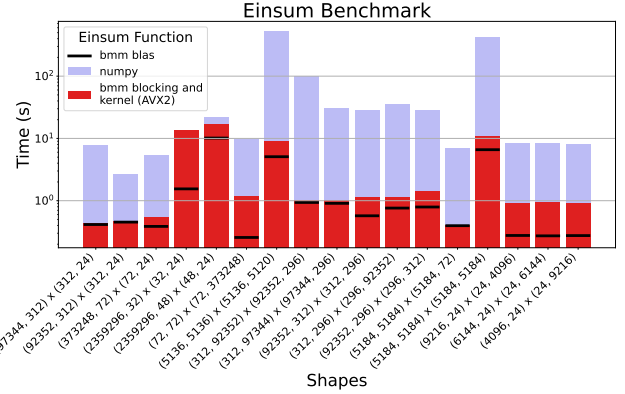


Figure 1: Performance comparison of einsum implementations. The benchmark shows significant performance gains for our implementation, especially for large input shapes we achieve up to 100x speedup over NumPy’s einsum. The benchmark instances are represented by their shapes when interpreted as matrices. Of the instances that have the same shapes when interpreted as matrices, only one is shown due to similar runtime behaviour.

resulting batch of matrices is then reshaped and transposed into the desired output shape. Let’s assume a simple tensor contraction:

$$C_{bm_1m_2n} \leftarrow A_{bm_1km_2} B_{bkn},$$

where b and k are the batch index and the contraction index. In order to employ batch matrix multiplication, the tensor A must be transposed to $\hat{A}_{bm_1m_2k} \leftarrow A_{bm_1km_2}$ and then reshaped to $\hat{A}_{b(m_1m_2)k}$, where the dimensions of m_1 and m_2 are combined into a single dimension (m_1m_2) . Eventually, the result $C_{b(m_1m_2)n}$ must be reshaped to the four-dimensional tensor $C_{bm_1m_2n}$.

In the general case, tensor contractions

$$C_{\pi^C(I_b \circ I_m \circ I_n)} \leftarrow A_{\pi^A(I_b \circ I_m \circ I_k)} B_{\pi^B(I_b \circ I_n \circ I_k)},$$

must be transposed and reshaped to fit the BMM scheme in the same way:

$$C_{I_b \circ I_m \circ I_n} \leftarrow A_{I_b \circ I_m \circ I_k} B_{I_b \circ I_k \circ I_n}$$

where I_b, I_k represent index groups of batch and contraction indices, $I \circ J$ denotes the concatenation of index groups and π^C, π^A, π^B stand for arbitrary permutations of indices. After transposition, the index groups I_b, I_k, I_m, I_n must be reshaped to a single dimension as in the example above. The resulting batch of matrices $C_{I_b \circ I_m \circ I_n}$ must be reshaped and transposed to fit the desired output shape $C_{\pi^C(I_b \circ I_m \circ I_n)}$. The process of transposition and index grouping is illustrated in Fig. 2. Summed indices occurring in only one tensor are trivially removed in a first step. If there are no batch dimensions,

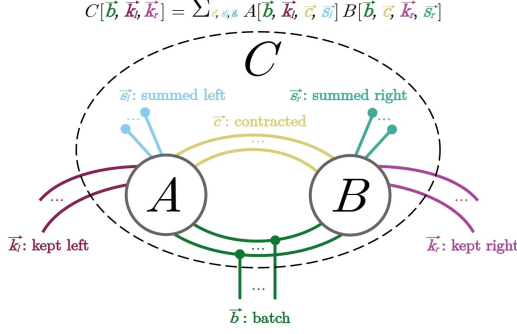


Figure 2: Index grouping of tensor contraction with a batch dimension. This figure is adapted from einsum_bmm.

GEMM (General Matrix Multiplication) can be used, or the batch dimension can be set to one.

1.2 Related Work

Since einsum was released by NumPy its application has grown rapidly and became more demanding. Python libraries like `opt_einsum` provided relief by finding optimal contraction orders for multi-contractions and calling specialized routines like BLAS for many operations [7].

In order to re-implement BMM in C++, we conducted some previous research on high-performance matrix multiplication [1, 6, 8, 10]. In their work Goto and van de Geijn [1] analyze different ways of reducing large GEMMs to smaller kernel multiplications, making the computation more cache-friendly. Their work led to the BLAS implementation GotoBLAS and inspired other popular BLAS implementations such as OpenBLAS [11].

1.3 Our Contributions

Our Python library provides a faster implementation of the einsum functionality than NumPy’s implementation in most cases, with a peak speedup of up to 100 times (Fig. 1). Although we provide an optimized version that is compatible with most hardware architectures, we also provide a performance portable version using the AVX2 vector instruction set. Furthermore, we present insights into the effect of various techniques such as kernelization, blocking, and parallelization.

1.4 Outline

Next, we describe different implementations of our core BMM functions and explain the techniques used. Finally, we compare our implementations to NumPy’s einsum function using a comprehensive benchmark set and conclude our work.

2 IMPLEMENTATION

Our main work consists of implementing a fast BMM for large tensors of double precision values in C++. As input we assume batches of matrices A and B of sizes $b \times m \times k$ and $b \times k \times n$.

The naive implementation uses four *for*-loops. Choosing the loop order so that the innermost loops have the smallest memory stride, we loop first over the batch dimension, then over m , then over the shared dimension k , and finally over n with *stride 1*. There

are two bottlenecks with this approach: CPU and memory access. We address the former in section 2.1 with a micro kernel that uses vector instructions to make multiplication of small matrices highly efficient. The latter is addressed by splitting matrices into smaller sub-matrices that can reside at different levels of the cache hierarchy [1, 8], see section 2.2. Performance is then further improved by parallelizing the code, as explained in section 2.3. Kernelization, blocking, and parallelization all improve the performance of GEMM, regardless of the batch dimension.

2.1 Kernelization

The micro kernel (or kernel) uses vectorization to provide highly optimized code that updates a sub-matrix of size $h \times w$ of C . It takes as input h rows and $r - l$ columns from A and $r - l$ rows and w columns from B . That is, it uses matrix slices with indices l to r in the shared dimension. For now we assume $l = 0$ and $r = m$, in section 2.2 this will be replaced according to the block size.

In practice, kernels are normally written and fine-tuned in assembler. For simplification we used OMP and vector intrinsics to design the following two kernels:

- (1) AVX2 kernel: Hardware specific, uses AVX2 vector intrinsics explicitly and is performance portable.
- (2) OMP kernel (e. g. general kernel): Vectorization using the `#pragma omp simd` compiler directive. This should be compatible with most hardware architectures but is not performance portable.

The basic structure of our AVX2-kernel 1 is shown in algorithm 1, adapted from [6]. The AVX2 instruction set provides 16 registers of size 256-bit each. Since our BMM works with doubles, four elements can be stored in a vector register. We use a temporary vector t to store the $h \times w$ elements, which are used to update C at the end. To perform the multiplication, four elements of B are loaded and multiplied by one element of A each, using the `m256d_broadcast_sd` intrinsic. Multiplication itself is done using a `fused multiply add` (`fma`) instruction. For the algorithm, we assume that A , B and C are 64-byte aligned in memory. This algorithm can now be optimized

Algorithm 1 Kernel

```

1: procedure KERNEL( $A, B, C, batch\_dim, a\_rows, b\_cols, a\_cols, a\_idx, b\_idx, l, r, h, w$ )
2:    $w_l \leftarrow w/4$  ▷ Four doubles in one vector register
3:    $t \leftarrow$  vector of _m256d with size  $h \times w_l$ 
4:    $zero \leftarrow \_m256d\_setzero\_pd()$ 
5:   for  $i \leftarrow 0$  to  $h \times w_l - 1$  do
6:      $t[i] \leftarrow zero$  ▷ Initialize result registers
7:   end for
8:    $offsetA \leftarrow (batch\_dim \times a\_rows + a\_idx) \times a\_cols + l$ 
9:    $offsetB \leftarrow batch\_dim \times a\_cols \times b\_cols + b\_idx + l \times b\_cols$ 
10:  for  $k \leftarrow l$  to  $r - 1$  do ▷ Iterate over contracting dimension
11:    for  $j \leftarrow 0$  to  $w_l - 1$  do
12:       $b0 \leftarrow \_m256d\_load\_pd(B[offsetB + j \times l])$ 
13:      for  $i \leftarrow 0$  to  $h - 1$  do
14:         $a0 \leftarrow \_m256d\_broadcast\_sd(A[offsetA + i \times a\_cols])$ 
15:         $t[i \times w_l + j] \leftarrow \_m256d\_fmadd\_pd(a0, b0, t[i \times w_l + j])$ 
16:      end for
17:    end for
18:     $offsetA \leftarrow offsetA + 1$ 
19:     $offsetB \leftarrow offsetB + b\_cols$ 
20:  end for ▷ Write results from  $t$  to  $C$ 
21:   $offsetC \leftarrow (batch\_dim \times a\_rows + a\_idx) \times b\_cols + b\_idx$ 
22:  for  $i \leftarrow 0$  to  $h - 1$  do
23:    for  $j \leftarrow 0$  to  $w_l - 1$  do
24:      for  $k \leftarrow 0$  to 3 do
25:         $C[offsetC + i \times b\_cols + j \times 4 + k] \leftarrow C[\dots] + t[i \times w_l + j][k]$ 
26:      end for
27:    end for
28:  end for
29: end procedure
    
```

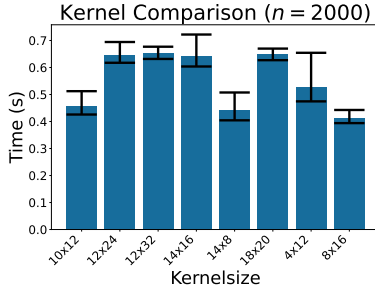


Figure 3: Runtime for BMM using different kernel sizes $h \times w$ on square matrices of size $n = 2000$. Each kernel was tested for all combinations of block sizes $b_1, b_2, b_3 \in 120, 240, 480$. The height of the bars corresponds to the average runtime over all block size combinations, the error bar shows the minimum and maximum runtime.

by unrolling the two innermost loops that iterate over h and w . This takes advantage of Instruction-Level Parallelism (ILP). Both the *load* and *broadcast* instructions have a latency of 7 and a throughput of < 0.5 , for *fma* these values are 4 and 0.5. Pipelining these instructions should speed up the algorithm, because the instructions can be executed simultaneously on the same CPU core.

To determine an optimal kernel size, we have three criteria:

- (1) Register spill: AVX2 uses 16 YMM registers. To minimize the time needed to move elements to the registers, the number of registers used by our kernel r should not exceed 16. $\rightarrow 16 \geq r = \frac{hw+w}{4} + 1$
- (2) I/O efficiency: To compute $h \cdot w$ elements of C we need $h \cdot (r - l)$ elements from A and $w \cdot (r - l)$ elements from B . The chosen kernel sizes should maximize the output-input ratio: $\max \frac{h \cdot w}{h + w}$.
- (3) ILP: The number of pipelined calls should be greater than or equal to the ratio of latency to throughput for each intrinsic. For the *fma* operation we get $8 \leq \#fma\text{-calls} = \frac{h \cdot w}{4}$.

It is not possible to write a kernel that fully satisfies all the criteria. Therefore, we experimented with reasonable kernel sizes and chose the one that worked best on our system. In our case, a kernel with $h = 8$ and $w = 16$ gave the best overall performance, see Fig. 3. The chosen kernel needs 37 registers, has an I/O efficiency of 5.3 and pipelines four *fma*-calls. Our test environment (see section 3) might have access to more than 16 registers, which could explain this outcome. In general, this kernel seems to strike a balance between the different criteria.

The OMP kernel follows the same structure as the AVX2 kernel. However, instead of explicitly vectorizing and unrolling loops, the compiler directive `#pragma omp simd` is used. This encourages the compiler to use SIMD instructions when available. As these are AVX2 instructions on our system, the same criteria and results apply to the choice of kernel size. Because we optimized the kernel size for our system, performance may be degraded on other systems. However, the kernel will still work correctly.

2.2 Cache Friendly Code With Blocking

As the kernel is designed to optimize CPU efficiency, our BMM implementation is bandwidth constrained. Therefore, we introduce

blocking techniques to make optimal use of the cache hierarchy. We first select a block of columns b_3 of B , then a block b_1 of the shared dimensions (columns of A and rows of B). This gives a sub-matrix B_{i1} of B of size $b_1 \times b_3$. A *pack*-operation copies and aligns B_{i1} and returns a smaller matrix B_p . Similarly, we select a sub-matrix A_{i2} of A of size $b_2 \times b_1$ and pack it into a smaller matrix A_p . The block sizes b_1, b_2, b_3 are chosen so that B_p fits in the L3 cache and A_p in the L2 cache. Next, we partition A_p and B_p further according to the micro kernel sizes h and w . We want each pair of $h \times b_1$ and $b_1 \times w$ elements to fit into the L1 cache for the execution of the micro kernel. Because the slices of A_p are accessed faster than of B_p , it is beneficial to store A_p in the L2 cache and B_p in L3. The kernel execution then takes place in the registers. The different blocking levels result in five loops around the kernel or six loops considering the batch dimension as well. This structure is known as the *BLIS-approach* [8, 10] and is illustrated in Fig. 4.

When choosing a kernel, we ran each candidate on different block size combinations. The chosen kernel performed best with $b_1 = 240, b_2 = 120$ and $b_3 = 120$, which we refined through further testing to $b_1 = 220, b_2 = 90$ and $b_3 = 90$. These block sizes are used later in our experiments.

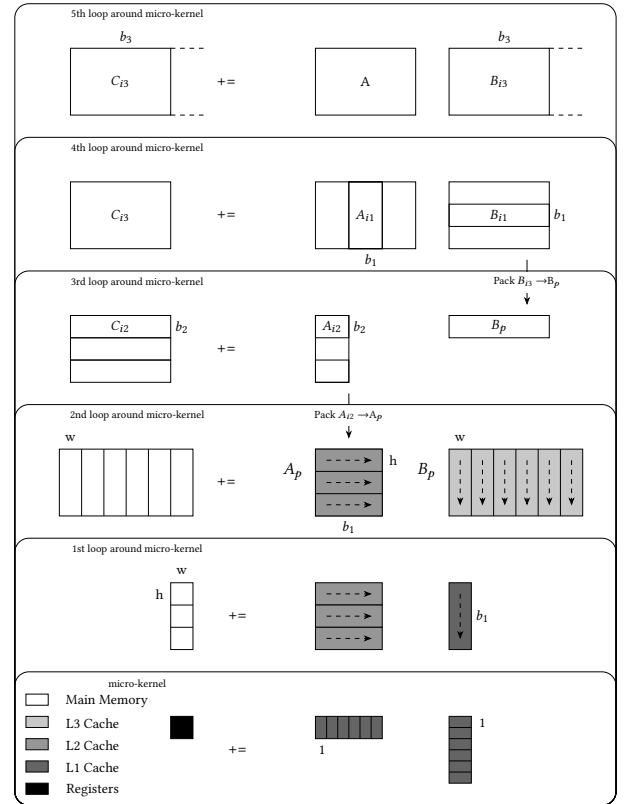


Figure 4: The structure of blocked matrix multiplication. Adapted from [2].

2.3 Parallelization

To take full advantage of modern multi-core processors, parallelization is key. OMP makes it very easy to parallelise independent iterations using the `#pragma omp parallel` for compiler directive. In

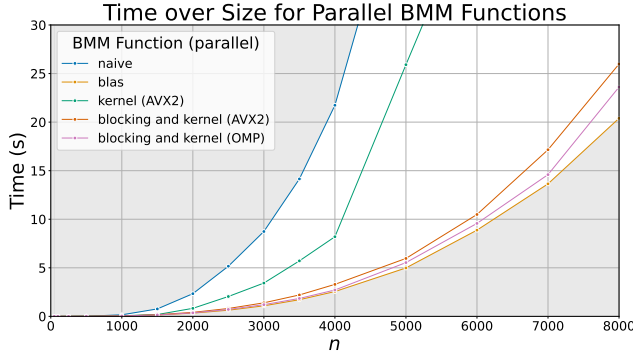


Figure 5: Runtime over square matrix size with batch dimension $bd = 4$ for parallelized BMM functions.

the case of BMM, possible race conditions need to be considered when parallelizing the contracting dimension k , since the same position in the resulting matrix can be written simultaneously. The other dimensions b , m and n are safe to parallelise.

Our tests suggest that parallelizing the outermost loops outperforms parallelizing the innermost loops. This might be expected differently, since our system (see section 3) has only one L3 cache [8]. However, the greater overhead of parallelizing these inner loops may explain why this is not advisable. In order to parallelize the three outermost loops, by using the *collapse(3)* keyword, blocking the the loop over the shared dimension k must be done thread-safe. We achieved this by assigning a local result matrix to each tread. These result matrices are combined after the computation using the compiler directive *#pragma omp critical* within the parallel region.

If the matrix dimensions do not align with the block sizes, the workers may receive chunks of varying sizes. Thus, we used *sched-ule(dynamic)* for the parallel region.

3 EXPERIMENTS

All experiments were done on a laptop running Arch Linux in its updated version as of February and March 2025 with 16 GB of RAM and an Intel Core i7-13700H CPU. For the compiler we used g++ 14.2.1 with the compiler flags “-Ofast -march=native -fopenmp”.

We analyze our implementation first by evaluating the performance of our BMM functions, using square input matrices of size $n \times n$. Second, we use the benchmark set from [9] to evaluate the performance of our einsum implementation using the BMM functions. All benchmarks were run on random matrices with several repetitions. We flushed the cache after each repetition to avoid reading data from the cache instead of main memory. Parallelization was used unless explicitly stated otherwise, since it gives a speedup for matrix sizes larger than $n = 100$ (see Fig. 6, section 3.1). To compare our BMM implementations, we used the OpenBLAS implementation of GEMM as a baseline. We extended it to loop over the batch dimension without parallelization, as the library function parallelizes internally.

3.1 BMM Performance

To analyze the performance of BMM, we used the naive implementation as lower and the BLAS library implementation as an upper baseline. We compare our BMM implementation in two stages of the optimization process. First, we apply only kernelization using our AVX2 kernel. Second, we combine blocking with both the AVX2 and OMP kernels. The results are shown in Fig. 5.

Kernelization reduces the runtime by a factor of about 2 compared to the naive BMM. Combined with blocking, our implementation is only slightly slower than the BLAS function. The OMP kernel performs slightly better on our system. The *simd* compiler directive encourages the compiler to vectorize the code using AVX2 instructions. Thus, the OMP kernel benefits from the same considerations for determining a good kernel size, and apparently the compiler manages to produce a slightly more efficient vectorization. If our BMM is used on a system that does not support AVX2 instructions, the performance of the OMP kernel may vary and a different kernel size may give better results. We optimized our im-

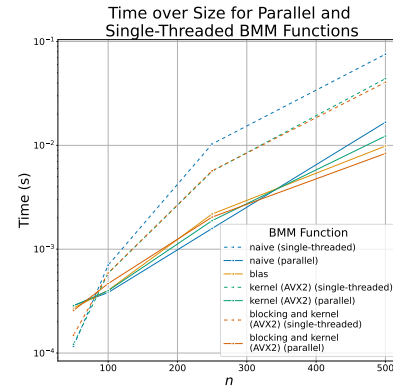


Figure 6: Runtime (logarithmic scale) over square matrix size with batch dimension $bd = 4$ for both single-threaded and parallel BMM.

plementation for large matrix dimensions. For matrix sizes $n \leq 250$, the naive implementation is faster than both our optimized BMM and the BLAS library function, see Fig. 6, as both kernelization and blocking introduce overhead. Aligning the matrices effectively results in copying each element into a smaller *packed* matrix. The additional loops, helper variables and padding add further overhead.

3.2 Einsum Benchmarks

The benchmark set contains 48 instances with different einsum format strings and variable sized input tensors. The set was originally compiled by Springer and Bienitinesi [9] containing a wide range of use cases. They used the benchmark to evaluate a GEMM-like tensor contraction approach. Therefore, we decided to add a batch dimension of 6 to all benchmark instances. Recall that the input tensors of the einsum expression are mapped to two batches of matrices for BMM, see section 1.1. Of instances which have the same shapes when interpreted as matrices, only one is shown in the figures 1 and 7 due to their very similar results. Both figures show the results of the same experiment. A complete table, including the

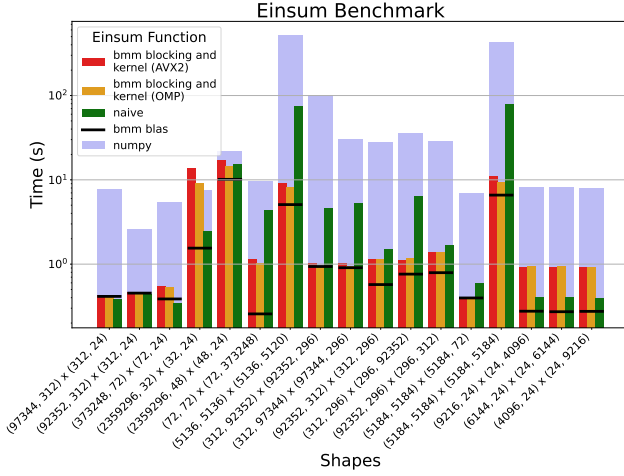


Figure 7: Performance comparison of einsum implementations. The benchmark shows significant performance gains of the BMM approach over NumPy’s einsum. Benchmark instances are represented by their shapes when interpreted as matrices. Of instances which have the same shapes when interpreted as matrices, only one is shown due to similar runtime behaviour.

einsum format strings and input tensor shapes, can be found in the appendix 4. In the following we use $b \times m \times k$ and $b \times k \times n$ to describe the shapes of the input matrices.

The evaluation of our einsum implementation shows that our BMM function outperforms the numpy einsum function by at least one order of magnitude in the majority of benchmark instances, as can be seen in Fig. 7. Einsum equipped with BLAS on the other side beats our implementation almost every time. Our BMM with the AVX2 and OMP kernels gave very similar results for all instances, as expected given the results in section 3.1. The benchmark results can be mainly divided into computationally demanding and memory bound instances. In cases of a large shared dimension (cases 7, 8, 9, 13 and 14 in Fig. 7) our optimized BMM functions consistently outperform the naive implementation, perform somewhat similar to BLAS and exceed NumPy’s einsum by up to 100 times. Here, the overhead of blocking, packing the matrices and using the kernel pays off.

Our optimized BMM functions excel slightly or are on par with the naive version in cases of a medium sized shared dimension (cases 1, 2, 10, 11 and 12 in Fig. 7). As the number of arithmetic operations decreases, the proportion of the computational overhead through blocking and packing grows. However, in cases with very large n , the naive implementation performs poorly, likely because the innermost loop iterates over n , which is not parallelized.

Eventually, our optimized BMM functions perform worst, when the shared dimension is smaller than 100. In case 4 even worse than NumPy’s einsum function, where both k and n are very small. Here, the overhead of blocking and packing cannot be amortized by computational gains anymore. Our kernel multiplies a $h \times (r-l)$ sub-matrix of A with a $(r-l) \times w$ sub-matrix of B . If $(r-l) = k < b1$, our kernel multiplies fewer elements in one call. Thus, more kernel calls are needed to perform the same number of multiplications, causing a

slowdown. In these cases however, the naive implementation mostly excels over the optimized BMM functions, in case 3 even slightly over BLAS. Thus, switching from the optimized BMM functions to the naive implementation in cases of low computational demand could improve the overall performance of our implementation.

4 CONCLUSIONS

In this project, we explored the acceleration of einsum by mapping tensor contractions to batched matrix multiplication and showed that this approach outperforms NumPy’s einsum function on our benchmark.

We also contributed our own implementation for BMM, which we optimized for large squarish matrices. In practice, our optimized implementations outperform NumPy’s einsum function by up to 100 times in these cases. When the computational demand is low, it can at least keep up with NumPy’s einsum. Switching to the naive BMM in these cases would make our implementation more competitive, but requires further investigation.

The bottleneck of our implementation seems to be the memory access, which could be improved by rethinking our parallelization technique and blocking sizes. Streaming the stores of the resulting matrix to bypass the cache hierarchy could also further improve the memory management. Finally, fine-tuning our kernel in assembly could take our implementation to the next level.

REFERENCES

- [1] Kazushige Goto and Robert A. van de Geijn. 2008. Anatomy of high-performance matrix multiplication. *ACM Trans. Math. Softw.* 34, 3, Article 12 (May 2008), 25 pages. <https://doi.org/10.1145/1356052.1356053>
- [2] Devin A. Matthews. 2018. High-Performance Tensor Contraction without Transposition. *SIAM Journal on Scientific Computing* 40, 1 (2018), C1–C24. <https://doi.org/10.1137/16M108968X> arXiv:1607.00291
- [3] Robert Peharz, Steven Lang, Antonio Vergari, Karl Stelzner, Alejandro Molina, Martin Trapp, Guy Van Den Broeck, Kristian Kersting, and Zoubin Ghahramani. 2020. Einsum Networks: Fast and Scalable Learning of Tractable Probabilistic Circuits. In *Proceedings of the 37th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 119)*, Hal Daumé III and Aarti Singh (Eds.). PMLR, 7563–7574. <https://proceedings.mlr.press/v119/peharz20a.html>
- [4] Tim Rocktäschel. 2018. Einsum is All you Need. <https://rocktai/2018/04/30/einsum/>. [Accessed 12-03-2025].
- [5] Noam Shazeer, Youlong Cheng, Niki Parmar, Dustin Tran, Ashish Vaswani, Penporn Koanantakool, Peter Hawkins, Hyounjoong Lee, Mingsheng Hong, Cliff Young, Ryan Sepassi, and Blake A. Hechtman. 2018. Mesh-TensorFlow: Deep Learning for Supercomputers. *CoRR abs/1811.02084* (2018). arXiv:1811.02084 <http://arxiv.org/abs/1811.02084>
- [6] Sergey Slotin. [n. d.]. Matrix Multiplication. <https://en.algorithmica.org/hpc/algorithms/matmul/>
- [7] Daniel G. A. Smith and Johnnie Gray. 2018. opt_einsum - A Python package for optimizing contraction order for einsum-like expressions. *Journal of Open Source Software* 3, 26 (Jun 2018), 753. <https://doi.org/10.21105/joss.00753>
- [8] Tyler M. Smith, Robert van de Geijn, Mikhail Smelyanskiy, Jeff R. Hammond, and Field G. Van Zee. 2014. Anatomy of High-Performance Many-Threaded Matrix Multiplication. *2014 IEEE 28th International Parallel and Distributed Processing Symposium* (May 2014), 1049–1059. <https://doi.org/10.1109/ipdps.2014.110>
- [9] Paul Springer and Paolo Bientinesi. 2018. Design of a High-Performance GEMM-like Tensor–Tensor Multiplication. *ACM Trans. Math. Softw.* 44, 3, Article 28 (Jan. 2018), 29 pages. <https://doi.org/10.1145/3157733>
- [10] Field G. Van Zee and Robert A. van de Geijn. 2015. BLIS: A Framework for Rapidly Instantiating BLAS Functionality. *ACM Trans. Math. Softw.* 41, 3, Article 14 (June 2015), 33 pages. <https://doi.org/10.1145/2764454>
- [11] Zhang Xianyi. 2012–2025. OpenBLAS: An optimized BLAS library. <http://www.openblaslib.org/OpenBLAS/>. [Accessed 13-03-2025].
- [12] Shi-Xin Zhang, Jonathan Allcock, Zhou-Quan Wan, Shuo Liu, Jiace Sun, Hao Yu, Xing-Han Yang, Jiezhong Qiu, Zhaofeng Ye, Yu-Qin Chen, Chee-Kong Lee, Yi-Cong Zheng, Shao-Kai Jian, Hong Yao, Chang-Yu Hsieh, and Shengyu Zhang. 2023. TensorCircuit: a Quantum Software Framework for the NISQ Era. *Quantum* 7 (Feb 2023), 912. <https://doi.org/10.22331/q-2023-02-02-912>

APPENDIX

Table 1: Full Benchmark Results (in seconds)

Tensor contraction	Tensor shapes	Matrix shapes	Naive	AVX2	OMP	BLAS	NumPy
zabc-zbda-zdc	6-312-312-312;6-312-24	6-97344-312;6-312-24	0.384	0.427	0.417	0.414	7.7
zabc-zdca-zbd	6-312-296-312;6-24-312	6-92352-312;6-312-24	0.435	0.465	0.457	0.453	2.608
zabcd-zdbea-zec	6-72-72-72-72;6-72-24	6-373248-72;6-72-24	0.342	0.546	0.534	0.386	5.382
zabcd-zdeca-zbe	6-72-72-72-72;6-24-72	6-373248-72;6-72-24	0.338	0.531	0.524	0.372	2.645
zabcd-zebad-zce	6-72-72-72-72;6-24-72	6-373248-72;6-72-24	0.363	0.56	0.549	0.405	2.514
zabcde-zefbad-zcf	6-48-32-32-48-32;6-24-32	6-2359296-32;6-32-24	2.484	13.661	9.204	1.548	7.606
zabcde-zecbfa-zfd	6-48-32-32-48-48;6-48-24	6-2359296-48;6-48-24	15.23	16.964	14.461	10.134	21.592
zabcde-zefcad-zbf	6-48-32-32-48-32;6-24-32	6-2359296-32;6-32-24	1.334	11.207	9.024	1.594	7.61
zabcd-zea-zebcd	6-72-72;6-72-72-72-72	6-72-72;6-72-373248	4.304	1.152	1.021	0.257	9.607
zabcd-zeb-zaecd	6-72-72;6-72-72-72-72	6-72-72;6-72-373248	4.499	1.309	1.28	0.453	17.188
zabcd-zec-zabed	6-72-72;6-72-72-72-72	6-72-72;6-72-373248	4.57	1.359	1.29	0.486	18.18
zab-zac-zcb	6-5136-5136;6-5136-5120	6-5136-5136;6-5136-5120	75.336	9.109	8.263	5.085	517.895
zab-zacd-zdbc	6-312-296-312;6-312-296-296	6-312-92352;6-92352-296	4.653	1.017	0.946	0.937	100.187
zab-zcad-zdcb	6-312-312-312;6-312-312-296	6-312-97344;6-97344-296	5.31	1.013	0.934	0.907	30.068
zabc-zacd-zdb	6-312-296-312;6-312-296	6-92352-312;6-312-296	1.483	1.142	1.141	0.572	28.067
zabc-zad-zbdc	6-312-296;6-312-296-296	6-312-296;6-296-92352	6.375	1.119	1.177	0.761	35.483
zabc-zadc-zbd	6-312-296-296;6-312-296	6-92352-296;6-296-312	1.655	1.391	1.392	0.792	28.338
zabc-zadc-zdb	6-312-312-296;6-312-296	6-92352-312;6-312-296	1.586	1.398	1.397	0.802	76.743
zabc-zadec-zebd	6-72-72-72-72;6-72-72-72	6-5184-5184;6-5184-72	0.591	0.402	0.386	0.397	6.837
zabcd-zaebf-zdfce	6-72-72-72-72;6-72-72-72-72	6-5184-5184;6-5184-5184	77.95	10.912	9.348	6.591	426.004
zabcd-zaebf-zfdec	6-72-72-72-72;6-72-72-72-72	6-5184-5184;6-5184-5184	78.42	11.415	9.756	6.376	510.186
zabcd-zaecf-zbfde	6-72-72-72-72;6-72-72-72-72	6-5184-5184;6-5184-5184	79.382	11.822	10.398	7.189	426.895
zabcd-zaecf-zfbed	6-72-72-72-72;6-72-72-72-72	6-5184-5184;6-5184-5184	80.491	11.399	9.939	7.09	509.014
zabcd-zaedf-zbfce	6-72-72-72-72;6-72-72-72-72	6-5184-5184;6-5184-5184	78.784	11.83	10.378	7.572	429.752
zabcd-zaedf-zfbec	6-72-72-72-72;6-72-72-72-72	6-5184-5184;6-5184-5184	80.64	11.376	9.924	6.803	508.792
zabcd-zaefb-zfdce	6-72-72-72-72;6-72-72-72-72	6-5184-5184;6-5184-5184	80.44	11.849	10.395	7.269	507.356
zabcd-zaefc-zfbed	6-72-72-72-72;6-72-72-72-72	6-5184-5184;6-5184-5184	78.92	11.431	9.978	7.53	1320.266
zabcd-zaefb-zfdec	6-72-72-72-72;6-72-72-72-72	6-5184-5184;6-5184-5184	80.938	11.491	9.967	6.808	1301.077
zabcd-zaefc-zbfde	6-72-72-72-72;6-72-72-72-72	6-5184-5184;6-5184-5184	77.114	12.034	10.449	7.467	597.085
zabcd-zaefd-zfbec	6-72-72-72-72;6-72-72-72-72	6-5184-5184;6-5184-5184	76.95	11.437	9.973	6.927	1280.274
zabcd-zaefb-zdfce	6-24-16-24-24;6-24-16-16-16	6-9216-24;6-24-4096	0.407	0.92	0.934	0.276	8.194
zabcd-zaefb-zdfdec	6-24-16-24-16;6-24-16-24-16	6-6144-24;6-24-6144	0.407	0.924	0.933	0.273	8.199
zabcd-zaefc-zgfab	6-24-16-24-16;6-24-16-24-16	6-6144-24;6-24-6144	0.403	0.927	0.94	0.276	8.167
zabcd-zaefc-zgfb	6-24-16-24-24;6-24-16-16-16	6-9216-24;6-24-4096	0.404	0.92	0.931	0.279	8.189
zabcd-zaefc-zgfb	6-24-16-24-16;6-24-16-24-16	6-6144-24;6-24-6144	0.402	0.928	0.932	0.276	8.199
zabcd-zaefc-zgfb	6-24-16-24-16;6-24-16-24-16	6-6144-24;6-24-6144	0.403	0.923	0.936	0.277	8.166
zabcd-zaefc-zgfb	6-24-16-24-24;6-24-16-16-16	6-9216-24;6-24-4096	0.391	0.926	0.929	0.278	8.189
zabcd-zaefc-zgfb	6-24-16-24-16;6-24-16-24-16	6-6144-24;6-24-6144	0.393	0.917	0.932	0.275	8.2
zabcd-zaefc-zgfb	6-24-16-24-16;6-24-16-24-16	6-6144-24;6-24-6144	0.408	0.932	0.932	0.277	8.165
zabcd-zaefc-zgfb	6-24-16-24-16;6-24-16-24-16	6-6144-24;6-24-6144	0.392	0.937	0.928	0.278	8.008
zabcd-zaefc-zgfb	6-24-16-24-16;6-24-16-24-16	6-6144-24;6-24-6144	0.406	0.934	0.933	0.276	7.956
zabcd-zaefc-zgfb	6-24-16-16-16;6-24-16-24-24	6-4096-24;6-24-9216	0.397	0.911	0.914	0.275	7.976
zabcd-zaefc-zgfb	6-24-16-24-16;6-24-16-24-16	6-6144-24;6-24-6144	0.407	0.922	0.937	0.277	7.988
zabcd-zaefc-zgfb	6-24-16-24-16;6-24-16-24-16	6-6144-24;6-24-6144	0.392	0.925	0.926	0.276	7.997
zabcd-zaefc-zgfb	6-24-16-16-16;6-24-16-24-24	6-4096-24;6-24-9216	0.398	0.917	0.93	0.275	7.987
zabcd-zaefc-zgfb	6-24-16-24-16;6-24-16-24-16	6-6144-24;6-24-6144	0.391	0.916	0.935	0.277	7.992
zabcd-zaefc-zgfb	6-24-16-24-16;6-24-16-24-16	6-6144-24;6-24-6144	0.392	0.919	0.934	0.279	7.992
zabcd-zaefc-zgfb	6-24-16-16-16;6-24-16-24-24	6-4096-24;6-24-9216	0.395	0.913	0.923	0.279	7.981